

Git y GitHub, explicado desde cero

Una guía para entenderlo bien la primera vez — y poder explicárselo después a alguien más.

1. Lo primero: Git y GitHub NO son lo mismo

Esta es la confusión número uno de todo principiante, así que empecemos por aquí.

	Git	GitHub
¿Qué es?	Un programa que corre en tu computadora	Un sitio web (un servicio en la nube)
¿Para qué sirve?	Llevar el historial de cambios de tus archivos	Guardar una copia de ese historial en internet y compartirla
¿Lo necesitas para programar?	Sí, si quieres control de versiones	No, es opcional — es solo un lugar donde alojar tu repositorio
¿Quién lo hizo?	Linus Torvalds (el mismo creador de Linux), en 2005	Una empresa (hoy propiedad de Microsoft)

Analogía simple: Git es como el “Control de cambios” de Word — un mecanismo que existe en tu computadora y que registra qué cambió, cuándo y quién lo hizo. GitHub es como Google Drive o Dropbox: un lugar en internet donde subes esos archivos (con todo su historial de cambios incluido) para tenerlos respaldados y compartirlos con otras personas.

Podrías usar Git toda tu vida sin tocar GitHub jamás — simplemente no tendrías una copia en la nube ni una forma fácil de compartir tu código con otros.

Existen alternativas a GitHub que hacen exactamente lo mismo (guardar repositorios de Git en la nube): **GitLab** y **Bitbucket** son las más conocidas. GitHub es solo la más popular.

2. Instalación: cómo empezar a trabajar

Aquí está el paso a paso completo, de cero a listo para hacer tu primer commit y subirlo a GitHub.

2.1 Instalar Git en tu computadora

Windows: 1. Descarga el instalador desde git-scm.com. 2. Ejecuta el instalador. Las opciones por defecto funcionan bien para empezar — no necesitas cambiar nada si eres principiante. 3. Esto instala, además de Git, una terminal llamada **Git Bash**, que te permite usar comandos de Git incluso si tu Windows no tiene una terminal tipo Linux.

macOS:

Opción 1: si tienes Homebrew instalado
`brew install git`

Opción 2: instalar solo las herramientas de línea de comandos de Apple
`xcode-select --install`

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install git
```

Verificar que quedó instalado (en cualquier sistema):

```
git --version
```

Si te muestra un número de versión (por ejemplo `git version 2.43.0`), ya quedó instalado correctamente.

2.2 Configurar tu identidad (solo una vez)

Git necesita saber quién eres, porque esa información queda registrada en cada commit que hagas:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_correo@ejemplo.com"
```

Usa el mismo correo que vayas a usar (o ya uses) en tu cuenta de GitHub — así los commits que subas se vincularán automáticamente con tu perfil.

Puedes confirmar que quedó bien guardado con:

```
git config --list
```

2.3 Crear tu cuenta en GitHub

1. Entra a github.com y da clic en **Sign up**.
2. Registra un correo, un nombre de usuario y una contraseña.
3. Con eso ya tienes un perfil donde vivirán tus repositorios remotos.

2.4 Conectar tu Git local con tu cuenta de GitHub

Este es el paso donde más se atoran los principiantes, así que vale la pena explicarlo bien: **desde 2021, GitHub ya no acepta tu usuario y contraseña normales** para operaciones de Git por terminal (push, pull, etc.), por seguridad. Necesitas una de estas dos formas de autenticarte:

Opción A — Token de acceso personal (HTTPS, la más sencilla para empezar): 1. En GitHub: Settings → Developer settings → Personal access tokens → Generate new token. 2. Eliges los permisos (para empezar, con permisos de repo es suficiente) y generas el token. 3. **Cópialo de inmediato** — GitHub solo te lo muestra una vez. 4. La primera vez que hagas git push, la terminal te pedirá usuario y contraseña: pones tu usuario de GitHub, y como “contraseña” pegas el **token** (no tu contraseña real). 5. Git (o el sistema operativo) guarda ese token después de la primera vez, así que no lo tendrás que volver a escribir en cada push.

Opción B — Llave SSH (la que prefieren muchos programadores con experiencia):

```
ssh-keygen -t ed25519 -C "tu_correo@ejemplo.com" # genera un par de llaves
cat ~/.ssh/id_ed25519.pub # muestra tu llave pública
```

Copias esa llave pública y la agregas en GitHub: Settings → SSH and GPG keys → New SSH key. A partir de ahí, usas URLs que empiezan con git@github.com:... en lugar de https://github.com/..., y no te vuelve a pedir contraseña ni token.

Recomendación para empezar: usa la Opción A (token). Es más simple de entender al inicio; puedes migrar a SSH más adelante si te sientes cómodo.

2.5 Conectarlo con VS Code (el IDE del curso)

VS Code detecta automáticamente que tienes Git instalado y activa el panel de **Control de código fuente** (el ícono de la ramita, en la barra lateral izquierda) sin que tengas que configurar nada extra.

Para conectar tu cuenta de GitHub directamente: 1. En VS Code, da clic en el ícono de cuenta (esquina inferior izquierda) y elige **Sign in with GitHub**. 2. Se abre tu navegador para autorizar el acceso. 3. Con eso, VS Code puede hacer push/pull sin pedirte el token cada vez.

Extensión útil (opcional pero recomendada): **GitHub Pull Requests and Issues**, que te permite crear y revisar Pull Requests sin salir de VS Code.

2.6 Prueba rápida de que todo funciona

Un mini-checklist para confirmar que la instalación completa quedó lista:

```
mkdir prueba-git && cd prueba-git
git init
echo "Hola mundo" > prueba.txt
git add .
git commit -m "Primer commit de prueba"
```

Luego, en GitHub, crea un repositorio vacío (botón **New**), copia la URL que te da, y:

```
git remote add origin https://github.com/tu-usuario/prueba-git.git
git push -u origin main
```

Si al recargar la página del repositorio en GitHub ves tu archivo `prueba.txt`, **todo quedó instalado y conectado correctamente**.

2.7 “Me marca error en `git push -u origin main`” — diagnóstico a prueba de errores

Este es el tropiezo más común de quien conecta Git con GitHub por primera vez. Antes que nada, corre estos tres comandos — con lo que devuelvan casi siempre se identifica la causa:

```
git branch          # ¿cómo se llama tu rama actual?
git remote -v       # ¿a qué URL está conectado "origin"?
git log --oneline   # ¿ya tienes al menos un commit?
```

Causa #1 (la más común, por mucho): el nombre de la rama no coincide. `git init` crea tu rama local llamada **master** por defecto, salvo que tengas configurado lo contrario — pero GitHub, desde 2020, usa **main** por defecto en repositorios nuevos. Si le pides a Git que suba **main** y tu rama local se llama **master**, verás:

```
error: src refspec main does not match any file(s) known to it
```

Solución: fuerza el nombre de tu rama antes del push (funciona sin importar cómo se llamaba antes):

```
git branch -M main
git push -u origin main
```

Causa #2: el repositorio de GitHub no está realmente vacío. Si al crearlo marcaste “Add a README file” (o `.gitignore` o licencia), GitHub ya generó un primer commit ahí, sin relación con tu historial local. Verás:

```
! [rejected]        main -> main (fetch first)
hint: Updates were rejected because the remote contains work that you do
not have locally.
```

Solución A (la más simple): borra ese repositorio en GitHub y créalo de nuevo **sin marcar ninguna casilla**, para que quede completamente vacío. Solución B (si no quieres borrarlo): combina los historiales:

```
git pull origin main --allow-unrelated-histories
git push -u origin main
```

Causa #3: falla de autenticación.

remote: Support for password authentication was removed on August 13, 2021.

fatal: Authentication failed

GitHub ya no acepta tu contraseña normal por terminal. Si te pide usuario y contraseña, en el campo de “contraseña” debes pegar tu **token de acceso personal** (sección 2.4), no tu contraseña real de GitHub.

Causa #4: la URL del remoto está mal o no se agregó.

fatal: 'origin' does not appear to be a git repository

o

fatal: remote origin already exists

Solución:

```
git remote -v                                # revisa qué hay configurado
git remote remove origin                     # si está mal, lo quitas
git remote add origin https://github.com/tu-usuario/tu-repo.git # y lo agregas bien
```

Causa #5: Git sigue usando un token viejo que ya borraste (guardado en caché).

remote: Invalid username or token. Password authentication is not supported for Git operations.

fatal: Authentication failed

Esto pasa cuando generaste un token nuevo en GitHub, pero tu sistema todavía tiene guardado el token **anterior** y lo sigue reenviando sin preguntarte. Hay que borrar esa credencial guardada para que Git vuelva a pedirla:

- **Windows:** Inicio → “Administrador de credenciales” → pestaña “Credenciales de Windows” → busca git:https://github.com → Quitar.
- **macOS:** app Acceso a Llaveros → busca github.com → elimina la entrada de contraseña de Internet.
- **Linux:** revisa git config --global credential.helper. Si es store, borra la línea de github.com en ~/.git-credentials. Si es cache, corre git credential-cache exit.

Después de borrar la credencial vieja, vuelve a intentar git push: te pedirá usuario y contraseña de nuevo, y ahí pegas el token **nuevo**.

¿No encuentras la credencial en el Administrador de Windows? A veces Git Credential Manager la guarda dentro de la subsección “Credenciales genéricas”, colapsada dentro de la pestaña “Credenciales de Windows” — fácil de pasar por alto. En vez de buscarla a mano, es más confiable pedirle a Git que la borre directamente:

```
git credential-manager github logout
```

Si ese comando no existe en tu versión de Git, usa el método universal (funciona sin importar dónde esté guardada):

```
git credential reject
```

y cuando la terminal quede esperando texto, escribe:

```
protocol=https  
host=github.com
```

seguido de una línea vacía y luego **Ctrl+Z** + Enter para terminar. Después de esto, `git push` debería pedirte credenciales frescas.

Desbloqueo rápido alternativo (sin tocar el Administrador de credenciales), tecleado directo en la terminal, sin guardarlo con `git remote set-url`:

```
git push https://tu-usuario:TU_TOKEN_NUEVO@github.com/tu-usuario/tu-repo.  
git main
```

El procedimiento completo, en el orden que evita las causas más comunes de una sola pasada:

<pre>git log --oneline commit</pre>	<i># 1. confirma que ya tienes al menos un</i>
<pre>git branch -M main</pre>	<i># 2. fuerza que tu rama se llame "main"</i>
<pre>git remote -v L correcta</pre>	<i># 3. revisa que "origin" apunte a la UR</i>
<pre>git push -u origin main</pre>	<i># 4. empuja tus cambios</i>

Si después de esto el error persiste, el mensaje de error exacto (copiado tal cual) casi siempre indica la causa con precisión, aunque no sea ninguna de las anteriores.

3. ¿Qué problema resuelve Git?

Imagina que estás escribiendo un ensayo en Word y vas guardando versiones: ensayo.docx, ensayo_v2.docx, ensayo_v2_final.docx, ensayo_v2_final_YA.docx... Es caótico, ocupa espacio, y si alguien más edita el archivo al mismo tiempo que tú, uno de los dos pierde su trabajo.

Git resuelve esto de forma elegante:

- Guarda el historial completo de cambios **dentro de una sola carpeta**, sin necesidad de duplicar archivos.
 - Te permite **regresar a cualquier versión anterior** en cualquier momento.
 - Permite que **varias personas trabajen en el mismo proyecto** sin pisarse el trabajo entre sí.
 - Te dice exactamente **qué cambió, cuándo y quién lo hizo**, línea por línea si es necesario.
-

4. Los conceptos fundamentales (con analogías)

Antes de ver comandos, hay que entender el vocabulario. Estos son los conceptos que forman la base de todo lo demás:

Repositorio (repo)

La carpeta de tu proyecto, pero con el historial de cambios activado. Cuando “inicializas” un repositorio, le estás diciendo a Git: “a partir de ahora, lleva un registro de todo lo que pase aquí”.

Commit

Una “fotografía” del estado completo de tu proyecto en un momento dado, acompañada de un mensaje que explica qué cambió. Es la unidad básica del historial de Git.

Analogía: si tu proyecto fuera una película, cada commit sería un fotograma. Puedes regresar a ver cualquier fotograma anterior cuando quieras.

Staging area (área de preparación)

Un paso intermedio entre “hice cambios en mis archivos” y “guardé esos cambios en un commit”. Te permite elegir **cuáles** cambios quieres incluir en el próximo commit, en lugar de guardar todo de golpe.

Analogía: es como armar una caja para enviar por correo. Primero eliges qué cosas van dentro de la caja (staging), y luego sellas la caja (commit).

Working directory (directorio de trabajo)

Los archivos tal como están en tu computadora en este momento, con los cambios que aún no has guardado en ningún commit.

Branch (rama)

Una línea de desarrollo independiente. Por defecto, todo repositorio tiene una rama principal (antes se llamaba master, hoy es más común main). Puedes crear una rama

nueva para probar algo sin arriesgar el código que ya funciona, y si el experimento sale bien, la “fusionas” (*merge*) de vuelta a la rama principal.

Analogía: es como escribir un borrador en una hoja aparte antes de pasarlo en limpio al cuaderno bueno. Si el borrador no funciona, simplemente lo desechas; el cuaderno bueno nunca se vio afectado.

Remoto (remote)

La versión de tu repositorio que vive en un servidor externo, como GitHub. Tu computadora tiene una copia local; GitHub tiene una copia remota. Por convención, al remoto principal se le llama *origin*.

5. Ramas a fondo: el concepto que más cuesta al principio

4.1 Qué es una rama, en realidad

Técnicamente, una rama **no es una copia de tus archivos**. Es solo una **etiqueta que apunta a un commit** — un simple nombre que Git usa para saber “hasta aquí llega esta línea de trabajo”. Cuando haces un nuevo commit estando en una rama, la etiqueta simplemente se mueve para apuntar al commit más reciente.

Esto es importante porque explica por qué crear una rama en Git es **instantáneo** y casi no ocupa espacio: no se están duplicando archivos, solo se está creando un nuevo “marcador” sobre el historial que ya existe.

```
          C4 (feature)
          /
C1 --- C2 --- C3 (main)
```

Aquí, *main* y *feature* comparten el mismo historial hasta C3. A partir de ahí, cada rama sigue su propio camino. Si en algún momento *main* también avanza (por ejemplo, alguien corrige un error urgente), las dos ramas se separan más:

```
          C4 --- C5 (feature)
          /
C1 --- C2 --- C3 --- C6 (main)
```

4.2 ¿Por qué existen las ramas?

Porque casi nunca quieres que el código en el que estás experimentando o construyendo algo a medias esté mezclado con el código que **ya funciona y que otros están usando**. Las ramas dejan la rama principal (*main*) siempre estable, mientras el trabajo en progreso ocurre en otro lado.

4.3 Comandos básicos

```
git branch                # ver en qué ramas existe el repositorio
(y en cuál estoy parado)
git branch nueva-funcion  # crear una rama nueva (sin moverte a ella
todavía)
git checkout nueva-funcion # moverte a esa rama
git checkout -b nueva-funcion # crear la rama Y moverte a ella, en un so
lo paso
git switch nueva-funcion  # forma moderna de moverte entre ramas (eq
uivalente a checkout)

git merge nueva-funcion   # (estando en main) trae los cambios de "n
ueva-funcion" hacia main
git branch -d nueva-funcion # borrar una rama que ya no necesitas (una
vez fusionada)
```

4.4 El flujo típico con una rama

```
git checkout -b agregar-login # 1. creo una rama para esta tarea espe
cífica
# ... edito archivos, hago commits normalmente en esta rama ...
git add .
git commit -m "Agrego formulario de login"

git checkout main             # 2. regreso a la rama principal
git merge agregar-login      # 3. incorporo los cambios de mi rama a
main
git branch -d agregar-login  # 4. borro la rama, ya cumplió su propó
sito
```

6. Cinco casos de uso reales de las ramas

Caso 1 — Desarrollar una función nueva sin arriesgar lo que ya funciona

Estás construyendo una app y el código en main ya funciona en producción. Quieres agregar un sistema de comentarios, pero es un cambio grande que tomará varios días.

```
git checkout -b feature/sistema-comentarios
```

Trabajas ahí con toda libertad — puedes romper cosas, probar ideas, hacer commits desordenados — sin que nada de eso afecte la versión que la gente está usando. Cuando el sistema de comentarios funciona completo, haces merge a main.

Caso 2 — Corregir un error urgente en producción (hotfix)

Tu app ya está publicada, y descubres un error grave: los usuarios no pueden iniciar sesión. Mientras tanto, en main también hay trabajo a medias de otra función que **no** quieres publicar todavía.

```
git checkout -b hotfix/error-login main    # rama corta, solo para el arreglo
# corriges el error
git commit -m "Corrijo validacion de contraseña"
git checkout main
git merge hotfix/error-login              # el arreglo se integra rápido
git push                                  # se publica el arreglo, sin
mezclar el resto del trabajo pendiente
```

La rama hotfix existe solo el tiempo necesario para resolver el problema puntual.

Caso 3 — Experimentar con una idea riesgosa (spike)

Quieres probar si cambiar de base de datos (por ejemplo, de SQLite a PostgreSQL) es viable para tu proyecto, pero no estás seguro de que valga la pena el esfuerzo.

```
git checkout -b experimento/postgresql
```

Si el experimento funciona, lo fusionas a main. Si no funciona o decides que no vale la pena, simplemente **borras la rama** (`git branch -D experimento/postgresql`) y es como si nunca hubiera pasado — main nunca se enteró de tu experimento.

Caso 4 — Trabajo en equipo, cada quien con su propia tarea

En un equipo de 3 personas construyendo la misma app: una persona hace el diseño de la interfaz, otra la lógica del carrito de compras, otra la conexión con la base de datos. Cada quien trabaja en su propia rama para no pisarse entre sí:

```
# Persona A
git checkout -b feature/interfaz

# Persona B
git checkout -b feature/carrito

# Persona C
git checkout -b feature/base-datos
```

Cada quien hace push de su rama a GitHub, abre un **Pull Request**, sus compañeros revisan el código, y solo entonces se fusiona a main. Así, main nunca tiene código a medias de nadie.

Caso 5 — Mantener varias versiones de un mismo producto (ramas de release)

Una empresa vende una app con clientes que usan la **versión 1.0** en producción, mientras el equipo ya está construyendo la **versión 2.0** con cambios grandes.

```
git checkout -b release/v1.0    # aquí solo se aplican correcciones menores a la v1.0
```

```
git checkout -b release/v2.0    # aquí avanza el desarrollo grande de la v2.0
```

Si un cliente de la v1.0 reporta un error, se corrige directamente en release/v1.0 y se publica sin tener que esperar a que la v2.0 esté terminada — y sin arrastrar a los clientes de la v1.0 funciones nuevas que aún no están listas.

7. Ya tiene el visto bueno: ¿cómo la integro a la rama principal?

Esta es la pregunta natural una vez que tu característica ya funciona y el equipo la aprobó. Hay dos caminos: hacerlo directo por línea de comandos (útil cuando trabajas solo), o hacerlo a través de GitHub con un Pull Request (lo normal cuando trabajas en equipo). Vale la pena entender ambos.

6.1 Camino 1 — Fusión directa por línea de comandos

```
git checkout main                # 1. te paras en la rama que va a recibir los cambios
git pull                         # 2. te aseguras de tener la versión más reciente de main
git merge nombre-de-tu-rama      # 3. traes los cambios de tu rama hacia main
git push                         # 4. subes esa fusión a GitHub para que los demás la vean
git branch -d nombre-de-tu-rama # 5. borras la rama local, ya cumplió su función
git push origin --delete nombre-de-tu-rama # 6. (opcional) borras también la rama en GitHub
```

¿Qué pasa exactamente en el paso 3? Depende de si main avanzó o no mientras tú trabajabas en tu rama:

- **Fast-forward:** si nadie más tocó main desde que creaste tu rama, Git simplemente “adelanta” la etiqueta de main hasta donde está tu rama. Es el caso más simple.

Antes: C1 --- C2 --- C3 (main)
 \
 C4 --- C5 (tu-rama)

Después: C1 --- C2 --- C3 --- C4 --- C5 (main, tu-rama)

- **Merge commit (fusión de tres vías):** si main sí avanzó mientras tanto (alguien más ya fusionó algo), Git crea un commit especial que une ambas líneas de historia.

Antes: C1 --- C2 --- C3 --- C6 (main)
 \
 C4 --- C5 (tu-rama)

Después: C1 --- C2 --- C3 --- C6 --- C7 (main) ← C7 es el "merge commit"

\ /
C4 ----- C5 (tu-rama)

6.2 Camino 2 — A través de un Pull Request en GitHub (lo recomendado en equipo)

Este es el flujo más común en el trabajo profesional, porque deja evidencia de la revisión y la aprobación:

1. Subes tu rama a GitHub (aún sin tocar main):

```
git push -u origin nombre-de-tu-rama
```

- En GitHub, abres un **Pull Request**: le dices “quiero integrar nombre-de-tu-rama a main”.
- Tus compañeros **revisan el código**, dejan comentarios, piden ajustes si hace falta.
- Cuando el equipo da el visto bueno (aprueba el PR), alguien presiona el botón **“Merge pull request”** directamente en la página de GitHub — no necesitas volver a tu terminal para eso.
- GitHub te ofrece tres formas de hacer esa fusión (las verás como opciones del botón):

Opción	Qué hace	Cuándo conviene
Merge commit	Igual que el “Camino 1”: conserva todos los commits de tu rama, tal cual, y agrega un commit de fusión	Cuando quieres conservar el historial detallado de cómo se desarrolló la función
Squash and merge	Junta todos los commits de tu rama en uno solo, con un mensaje limpio, antes de agregarlo a main	Cuando hiciste muchos commits “de prueba” en tu rama y quieres que main se vea ordenado
Rebase and merge	Coloca tus commits, uno por uno, justo después del último commit de main, sin crear un commit de fusión adicional	Cuando quieres un historial lineal, sin bifurcaciones visibles

6. Una vez fusionado, GitHub te ofrece un botón para **borrar la rama** automáticamente (ya cumplió su propósito).
7. En tu computadora, actualizas tu copia local para tener el resultado:

```
git checkout main
git pull
```

6.3 ¿Y si hay un conflicto?

A veces, tú y otra persona editaron **las mismas líneas** del mismo archivo en ramas distintas. Ahí Git no puede decidir solo cuál versión es la correcta, y te lo hace saber con un **conflicto de fusión** (*merge conflict*). No es un error grave, es normal — simplemente Git te pide que tú decidas.

Git marca el archivo así:

```
<<<<<<< HEAD
mensaje = "Hola desde main"
=====
mensaje = "Hola desde mi rama"
>>>>>>> nombre-de-tu-rama
```

Para resolverlo: 1. Abres el archivo y decides qué versión (o combinación de ambas) debe quedar. 2. Borrás las líneas <<<<<<<, ===== y >>>>>>> que Git agregó como marcadores. 3. Guardas el archivo, y le dices a Git que ya quedó resuelto: `bash git add archivo.py git commit -m "Resuelvo conflicto en mensaje de bienvenida"`

Si el Pull Request se está haciendo en GitHub, la plataforma también puede avisarte del conflicto e incluso, si es un conflicto simple, dejarte resolverlo directamente desde el navegador.

8. El flujo de trabajo básico de Git (local, sin ramas)

Este es el ciclo que vas a repetir una y otra vez cuando trabajas directamente sobre una sola rama:

1. Editas archivos en tu carpeta (working directory)
2. Eliges qué cambios quieres guardar (git add)
3. Guardas esos cambios como un commit (git commit)

En comandos:

```
git init # (solo una vez) convierte la carpeta e
n un repositorio
git add archivo.py # prepara un archivo específico para el
commit
```

```
git add . # prepara TODOS Los archivos modificados
git commit -m "Agrego la función de login" # guarda el commit con un mensaje descriptivo
```

Comandos para “mirar” el estado de las cosas, sin modificar nada:

```
git status # ¿qué archivos he cambiado? ¿cuáles están listos para el commit?
git log # historial de commits (quién, cuándo, qué mensaje)
git diff # ¿qué líneas exactas cambiaron, palabra por palabra?
```

Regla de oro: haz commits pequeños y frecuentes, con mensajes claros. Un commit como “cambios” no le dice nada a nadie (ni siquiera a ti mismo en un mes). Un commit como “Corrijo el cálculo de IVA en el carrito de compras” sí.

9. Lo contrario de `git init`: cómo dejar de usar Git en una carpeta

Con el tiempo puede que termines un proyecto y quieras que Git deje de monitorear esa carpeta — que vuelva a ser una carpeta normal en tu computadora, sin historial de versiones.

Todo lo que Git necesita para llevar el control de tu proyecto vive en **una sola carpeta oculta llamada `.git`**, en la raíz del repositorio. Borrar esa carpeta es todo lo que hace falta.

9.1 El procedimiento

1. Confirma dónde está la carpeta oculta:

```
ls -a
```

(En Windows con PowerShell: `Get-ChildItem -Force`. En el Explorador de archivos: activa “Elementos ocultos” en la pestaña Vista.)

2. Bórrala:

Sistema	Comando
macOS / Linux (terminal)	<code>rm -rf .git</code>
Windows (PowerShell)	<code>Remove-Item -Recurse -Force .git</code>
Windows (símbolo del sistema)	<code>rmdir /s /q .git</code>

3. Confirma que ya no es un repositorio:

```
git status
```

Debería responder fatal: not a git repository (or any of the parent directories): .git — eso confirma que la carpeta volvió a ser una carpeta normal.

9.2 Dos advertencias importantes

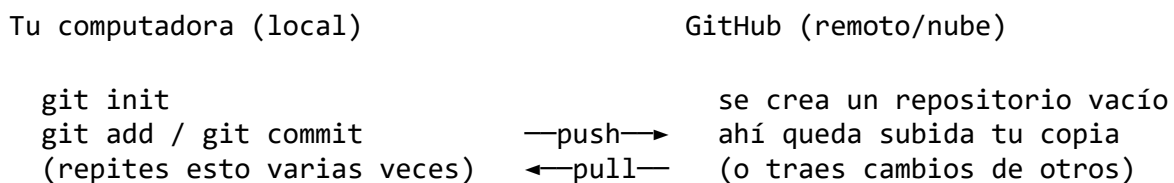
- **Es irreversible.** Al borrar .git desaparece todo el historial de commits, ramas y etiquetas que solo existían en tu computadora. Tus archivos de código quedan intactos — lo único que se pierde es el registro de cambios. Si crees que podrías necesitar ese historial después, copia la carpeta .git antes de borrarla, en vez de perderla para siempre.
- **No afecta a GitHub.** Si ya habías hecho push de ese proyecto, borrar .git localmente no borra nada en GitHub — el repositorio remoto sigue existiendo tal cual. Para eliminarlo también de ahí es una acción aparte, igualmente irreversible: Settings → General → Danger Zone → Delete this repository.

Nota: si en realidad solo quieres desconectar la carpeta de un repositorio de GitHub pero conservando tu historial local, eso es distinto: `git remote remove origin`. Borrar .git es para cuando quieres que la carpeta deje de tener cualquier rastro de Git.

10. ¿Dónde entra GitHub?

Hasta aquí, todo lo que vimos pasa **solo en tu computadora**. Nadie más puede ver tu historial de commits, y si tu disco duro muere, lo pierdes todo. Ahí es donde entra GitHub: es el lugar donde subes tu repositorio para tener una copia en la nube y compartirla.

El flujo completo con GitHub



Los comandos nuevos que se agregan

Comando	¿Qué hace?
<code>git clone <url></code>	Descarga una copia completa de un repositorio que ya existe en GitHub, con todo su historial
<code>git push</code>	Sube tus commits locales a GitHub
<code>git pull</code>	Descarga los commits nuevos que hay en GitHub y los mezcla en tu copia local

Comando	¿Qué hace?
<code>git remote add origin <url></code>	Conecta un repositorio local (ya existente) con uno vacío en GitHub

Dos formas de empezar un proyecto con GitHub

Opción A — Ya tienes el proyecto en tu computadora:

```
git init
git add .
git commit -m "Primera version"
git remote add origin https://github.com/tu-usuario/tu-repo.git
git push -u origin main
```

Opción B — El proyecto ya existe en GitHub y quieres traerlo a tu computadora:

```
git clone https://github.com/usuario/repo.git
```

Esto crea la carpeta automáticamente, con todo el historial incluido, ya conectada a GitHub.

11. Conceptos exclusivos de GitHub (no de Git)

Estos son conceptos que existen porque GitHub es una plataforma social/colaborativa, no porque Git los necesite:

Fork

Una copia completa de un repositorio ajeno, dentro de tu propia cuenta de GitHub. Se usa cuando quieres proponer cambios a un proyecto en el que no tienes permiso directo de escritura (por ejemplo, un proyecto de código abierto). Haces un fork, cambias lo que quieras en tu copia, y luego propones que esos cambios se incorporen al original.

Pull Request (PR)

Una propuesta formal de “oye, hice estos cambios, ¿los aceptas en tu proyecto?”. Es el mecanismo central de colaboración en GitHub: alguien revisa tus cambios, puede comentar línea por línea, pedir ajustes, y finalmente aprobarlos o rechazarlos.

Nota curiosa: el nombre viene de que técnicamente le estás “pidiendo” (request) al dueño del repositorio que haga un pull de tus cambios.

Issues

El sistema de “pendientes” de GitHub: reportar errores (*bugs*), proponer nuevas funciones, o discutir ideas — todo organizado como tickets, cada uno con su propia conversación.

README.md

El archivo que GitHub muestra automáticamente en la página principal de un repositorio. Es la “carta de presentación” del proyecto: qué hace, cómo instalarlo, cómo usarlo.

Repositorio público vs. privado

Público: cualquier persona en internet puede verlo (aunque no necesariamente editarlo). Privado: solo tú y las personas que invites pueden verlo.

12. Un día típico trabajando con Git y GitHub

Para amarrar todo, así se ve un flujo de trabajo real:

1. **Por la mañana**, traes los cambios más recientes que tus compañeros subieron:
2. **Trabajas** en tu código durante un rato, resolviendo una tarea específica.
3. **Guardas tu avance** en un commit (o varios, según lo vayas completando):

```
git pull
```

```
git add .  
git commit -m "Agrego validacion del formulario de registro"
```

4. **Subes** tus cambios a GitHub para que los demás los vean:
5. Si trabajas con ramas: creas una rama nueva para tu tarea, subes esa rama, y abres un **Pull Request** para que alguien revise tu código antes de integrarlo a la rama principal.

```
git push
```

13. Errores típicos de quien empieza (y cómo entenderlos)

Situación	Qué está pasando en realidad
“Hice <code>git push</code> y me dice rejected”	Alguien más subió cambios antes que tú; primero necesitas hacer <code>git pull</code> para

Situación	Qué está pasando en realidad
“No sé si mis cambios ya están guardados”	traerlos, y luego <code>git push</code> de nuevo
“Hice un commit con un error de dedo en el mensaje”	Corre <code>git status</code> : te dice exactamente qué archivos tienen cambios sin commit <code>git commit --amend</code> te permite corregir el mensaje del último commit
“Quiero deshacer cambios que aún no he guardado en un commit”	<code>git checkout -- archivo.py</code> regresa ese archivo a como estaba en el último commit
“Subí algo a GitHub por error y quiero borrarlo del historial”	Es posible, pero es más avanzado (<code>git revert</code> , <code>git reset</code>) — mejor prevenir con un buen <code>.gitignore</code>

14. Glosario rápido para tener a la mano

Término	En una frase
Repositorio	La carpeta del proyecto con el historial de Git activado
Commit	Una “fotografía” guardada del proyecto, con un mensaje explicativo
Staging area	Donde eliges qué cambios entran al próximo commit
Branch (rama)	Una línea de desarrollo independiente
Merge	Fusionar los cambios de una rama en otra
Merge conflict	Cuando Git no puede decidir solo cuál versión de una línea es la correcta y te pide que tú lo decidas
Squash	Juntar varios commits de una rama en uno solo antes de fusionarlos
Remote	La copia del repositorio que vive en un servidor (ej. GitHub)
Push	Subir tus commits locales al remoto
Pull	Traer los commits nuevos del remoto a tu copia local
Clone	Descargar una copia completa de un repositorio remoto
Fork	Tu propia copia, en tu cuenta, de un repositorio ajeno

Término	En una frase
Pull Request	Una propuesta de incorporar tus cambios al proyecto original
README	El archivo de presentación de un repositorio

15. Lo que hay que recordar si solo te queda una idea

Git es el motor que lleva el historial de cambios de tu proyecto, en tu propia computadora. **GitHub** es el lugar en internet donde guardas una copia de ese historial y lo compartes con otras personas.

Todo lo demás — commits, ramas, pull requests, forks — son piezas que existen para resolver un problema muy concreto: que el trabajo en equipo sobre el mismo código no sea un caos.